ArchIntel — Developer Guide and Internal Design

> **Phase 6 checkpoint:** Controlled source tasks and the pinned local Docker build sandbox are implemented. A controlled Java/Maven fixture passed a real offline, non-root, network-disabled build and test. Full Phase 6 acceptance remains open: customer artifact reproduction and broader characterization/security/lineage capabilities remain pending. See verification, user flow and developer handoff. Phase 7A now supports baseline/candidate evidence comparison, captured candidate artifacts and task-linked static re-analysis with structural differences. Refactor recipes, security scanning and final acceptance remain pending. See Phase 7 user flow and developer handoff.
>
> **Phase 5 update:** The Modernization intelligence workspace now creates immutable evidence-backed planning proposals, separate readiness/confidence, decisions, targets and dependency waves, with validation and actor-attributed review. Planning approval does not authorize code changes. See user flow, developer guide and verification.
>
> **Phase 4 evidence intelligence (4 October 2026):** Open Evidence & digital twin for version capture, reviewed domains, fact provenance, guided discovery and the persistent knowledge graph. See user flow, developer handoff and validation. Existing static/runtime workflows remain available.
>
> **Phase 3 runtime update (4 October 2026):** For current session setup, instrumentation preview/exclusions, strict identity declarations, quality diagnostics and recovery, read Phase 3 user flow and developer handoff. Validation results include a new real PetClinic capture; older results below remain historical.

Optional local-model setup and controls: Phase 2 user flow. Internal contracts and extension rules: Phase 2 developer guide. **Version 0.2 | Phase 2 AI gateway | 4 October 2026**

1. Handoff scope

This guide describes the implemented prototype, actual processing behavior, extension points, and production gaps. Read the user manual and verification report alongside it. The demo uses compiled synthetic Java and real analysis; it does not manufacture completed results.

Security implementation and extension rules are documented in PHASE-1-DEVELOPER-GUIDE.md.

2. Architecture

```
React / TypeScript / TanStack Query / Cytoscape
                  |
             FastAPI /api/v1
                  |
     PostgreSQL metadata and result snapshots
                  |
       Single-process analysis worker
                  |
    Validate archive -> extract application classes
                  |
          jQAssistant CLI 2.7.0
                  |
       Job-owned embedded Neo4j 5.26.5
                  |
       ExportGraph Java helper (Neo4j API)
```

```
                  |
       Normalize owned types and typed edges
                  |
     Separate class and package DiGraphs
                  |
   SCCs -> metrics -> domains -> issues -> readiness
                  |
      Domain-SCC condensation -> dependency waves
                  |
           Atomic final snapshot
                  |
    REST views / export / bounded evidence retrieval
```

The original standalone Bolt extraction spike was replaced with direct access to a per-job embedded store. Integration testing exposed a temporary Neo4j web-server lifecycle issue. The Java exporter now opens the completed scan store directly; no temporary web server or Bolt listener is required.

The local demo uses real PostgreSQL. SQLite is an explicitly selected test/convenience mode, never an automatic fallback when PostgreSQL fails. The API serves the built frontend on the same origin.

3. Code map

Module	Responsibility
`backend/app/main.py`	Factory, migrations, worker lifecycle, health, static frontend.
`config.py`, `models.py`, `schemas.py`	Settings, relational entities, request/response contracts.
`api.py`	Versioned endpoints, admission and result slicing.
`services/ingestion.py`	Safe ZIP handling, class extraction, hashing and cleanup.
`services/scanner.py`	Bounded trusted-tool subprocess orchestration.
`backend/scanner-java/ExportGraph.java`	Parameterized Neo4j queries and plain JSON export.
`services/graph.py`	Ownership filtering, typed deduplication, graph construction.
`services/cycles.py`, `metrics.py`	SCC-first cycle sampling and metric arithmetic.
`services/domains.py`, `issues.py`	Candidate boundaries, database evidence and rule findings.
`services/modernization.py`	Score components and dependency-first waves.
`services/engine.py`	Pure algorithm orchestration and snapshot creation.
`services/pipeline.py`	Worker, durable progress, atomic success and cleanup.
`services/assistant.py`	Bounded retrieval and evidence-only provider behavior.
`backend/alembic`	Versioned database migration.
`frontend/src`	Workspace, overview, explorer, insights, assistant and shared UI.
`frontend/e2e`	Live Chrome walkthrough and real upload tests.
`samples`	Synthetic sources plus compiled JAR/WAR/Boot fixtures.

Module	Responsibility
`scripts`	Setup, builds, start/stop, verification and document generation.

`.runtime` contains ignored downloaded runtimes; `.data` contains ignored database, jobs, logs and test artifacts. Neither belongs in source control.

4. Setup and operation

Prepared Windows x64 runtime: Node 22, workspace Python 3.12.10, Temurin JDK 21, jQAssistant 2.7.0, bundled Neo4j 5.26.5, PostgreSQL 18.4 via embedded-postgres. Exact installed versions and verified behavior are recorded in the verification report.

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/setup.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/start.ps1
```

Open `http://127.0.0.1:8000`; API documentation is `/docs` and schema is `/openapi.json`. Start-ArchIntel.cmd is the user-facing launcher. The launcher waits for an authenticated database query before starting the API, avoiding PostgreSQL recovery/startup races.

For frontend development:

```
cd frontend
npm ci --legacy-peer-deps
npm run dev -- --host 127.0.0.1
```

Vite runs on 5173 and proxies `/api` to localhost:8000. Rebuild before demonstrating the production-served page. After backend changes, run `scripts/restart-api.ps1`. Stop only when no jobs are active using `scripts/stop.ps1`; it preserves database contents. Recorded process identity is checked before terminating the API.

The included Docker configuration is an alternative developer deployment. Local Windows operation is the demonstrated path; do not infer Docker runtime verification from the existence of Dockerfiles.

5. Settings

Settings come from repository `.env`. Generated secrets must not be committed.

Setting	Purpose
`DATABASE_URL`	PostgreSQL+psycopg URL or explicit SQLite local/test URL.
`POSTGRES_PASSWORD`	Local PostgreSQL bootstrap credential.
`DATA_DIR`	Owned storage root; defaults to `.data`.
`JAVA_HOME`, `JQASSISTANT_HOME`	Override workspace runtime discovery.
`CORS_ORIGINS`	JSON array of local origins.
`EXTERNAL_PREFIXES`	Configurable namespaces excluded from scoring.
`MAX_UPLOAD_MB`, `MAX_EXPANDED_MB`	Upload and expansion limits.
`MAX_ARCHIVE_ENTRIES`, `MAX_CLASSES`, `MAX_EDGES`	Archive and extraction bounds.

Setting	Purpose
`SCAN_TIMEOUT_SECONDS`	Per scanner/export subprocess timeout.
`RETAIN_ARTIFACTS`	Default false; explicitly preserve uploads if needed.
`MAX_CYCLE_LENGTH`, `MAX_CYCLES_PER_SCC`, `LARGE_SCC_THRESHOLD`	Defaults 8, 20, 50.
`READINESS_WEIGHTS`	Four nonnegative components summing to one.
`LLM_URL`, `LLM_MODEL`	Legacy settings do not authorize dispatch. Use provider registration and workspace model policy; deterministic evidence remains available.

Ownership is established before prefix filtering. Do not blindly classify all `org.apache` or other familiar namespaces as third party: they may contain the user's application. Keep configuration and algorithm version with every snapshot. The launcher fixes PYTHONHASHSEED=0; Louvain uses seed 42.

6. Persistence

Alembic revision 0001 creates projects, applications, artifacts, and analysis_runs. Artifacts contain generated paths, logical names, SHA-256, size, validation manifest and availability. Runs contain status, progress, stage durations, timestamps, errors and a final JSON snapshot.

The snapshot contains classes/packages/domains, all graph levels, external references, metrics, cycles, issues, readiness, sequence, scanner metadata, settings and limitations.

Prototype tradeoff: graph-derived entities are immutable JSON results, not separate indexed relational tables. Slicing occurs in the API process. Before scaling, migrate to indexed run-owned result tables or a persistent normalized graph service while preserving export compatibility.

Existing records retain `tenant_id='local'`. Phase 1 now derives tenant identity from authentication and applies scoped ORM sessions plus parent ownership checks. See PHASE-1-DEVELOPER-GUIDE.md; production hosting and cross-process enforcement require further hardening. Some result schemas use typed envelopes with flexible dictionary payloads; harden these before promising a long-term external API contract.

Application startup runs migrations, not create_all. Unit/API tests use temporary SQLite through the same migration; the live demo uses PostgreSQL. Test new dialect-specific migrations on PostgreSQL.

7. Artifact and scanner lifecycle

Uploads are streamed to UUID-named files. Content-Length and streamed byte counts enforce limits. ZIP checks reject unsafe/drive/traversal paths, encrypted entries, symlinks, duplicate class paths, excessive entries, expansion and compression ratios. Class magic headers are checked.

Application classes are normalized from plain, WEB-INF/classes and BOOT-INF/classes layouts into `.data/jobs/<run-id>/classes`. Nested JARs are skipped; multi-release overrides are skipped and disclosed.

jQAssistant must use explicit `java:classpath` scope for extracted directories. The store URI must be an absolute normalized path: `./` path components caused Neo4j validation failures on Windows.

The scanner finishes before ExportGraph opens its Neo4j store. ExportGraph is compiled against the distribution libraries; rebuild it when changing scanner versions. It emits plain values, not Neo4j model objects. Query parameters include owned type names and limits.

Customer classes never enter the JVM execution classpath. Only trusted scanner/exporter tooling runs. Primitive pseudo-types and referenced-only classes cannot inflate application counts. Array dependencies normalize to component FQNs. Missing runtime/reflective edges are not evidence of independence.

Typed categories: METHOD_CALL, FIELD_TYPE, GENERIC_TYPE, ANNOTATION, INHERITANCE, INTERFACE, EXCEPTION, OTHER. Generic traversal depth is bounded. OTHER is retained only for a pair without a more precise extracted relation. The full dependency semantics are regression-tested on synthetic bytecode.

Temporary stores/classes are deleted after each attempt. Uploaded artifacts are deleted after successful processing by default; failed inputs remain available for retry. Cleanup verifies containment within the owned root. Logs expose operational stages and exception types, not artifact contents. Crash leftovers, failed inputs, and abandoned uploads need a production retention service.

8. Graphs, cycles and metrics

Direction: **dependent → dependency**. Deduplicate typed records by source/target/type. Algorithms count distinct internal source/target pairs, not invocation frequency or type multiplicity. Exclude class self-references. Preserve external references separately before scoring.

Package edges count distinct cross-package class pairs; intra-package references never create package self-loops. Class and package graphs are logically independent.

Compute strongly connected components first. Report components larger than one node. For SCCs ≤50 nodes, sample at most 20 cycles of length ≤8, requesting one extra to detect truncation. For larger components, skip enumeration while retaining complete membership and internal edge counts.

After domains exist, classify cross-domain clusters as critical, cross-package clusters high, same-package clusters medium; same-domain interface patterns may be down-weighted. These severities are versioned heuristics.

Ca and Ce count distinct outside source and target classes respectively. I=Ce/(Ca+Ce), zero on no coupling. A counts abstract OR interface once per type. D=abs(A+I−1). Isolated packages are flagged separately.

Class fan-in/out use distinct neighbours. P95 uses linear interpolation. God-class candidates require a cohort of at least five classes and two signals strictly above p95 among methods, fields and fan-out. Other rules inspect main-sequence deviation, unstable dependency direction, cross-domain ratios and controller boundary dependencies. Each issue includes evidence and recommended investigation.

9. Domains and database evidence

Candidate grouping uses meaningful package segments after the common root, avoiding generic layers such as service/repository. Seeded Louvain supplies fallback communities and agreement evidence. Package naming, agreement and annotations contribute to bounded confidence.

This is not trained ML or business ontology inference. Confidence is not calibrated. Every class has one candidate membership; IDs derive from sorted members. Manual/hybrid overrides require explicit future identity/provenance design.

Store raw internal/outgoing/incoming edge counts and outgoing cross-domain ratio.
JDBC/JPA/DataSource/JdbcTemplate/repository/entity references provide database indicators. Repository/entity links and shared consumer domains are structural candidates, not proof of shared runtime SQL or tables. Unknown data remains unknown.

10. Readiness and sequencing

Each assessment stores components, weights, risks, blockers, sequence and explanation.

Component	Formula
Cohesion	100 × internal/(internal+outgoing); 50 with no outgoing evidence.
Decoupling	100 × (1−outgoing cross-domain ratio).
Cycles	100 × (1−cyclic member fraction), minus 25 for cross-domain cycles, floored at zero.
Database	25 with shared repository candidates; 60 with structural signals; 50 if unknown.

Default weights: .25, .30, .25, .20. Difficulty: LOW ≥80; MEDIUM ≥60; HIGH ≥40; otherwise VERY_HIGH. Unknown database sensitivity varies that component from 0 to 100; it is not statistical confidence.

Incoming/outgoing pressure and boundary counts are exposed, but not all advisory signals receive independent weights in this prototype. Calibrate future changes against ground truth and bump algorithm versions.

Condense the domain graph into SCCs. Remove dependency-free sink groups in waves; rank by readiness within each wave. SCC members form coupled remediation groups and must not be presented as independent sequential extractions.

11. Assistant

ArchitectureContextBuilder retrieves a bounded fact set for a named domain or supported question category. The artifact and entire graph are never sent to a provider.

LOCAL_EVIDENCE is deterministic. Optional Ollama evidence selection now uses the policy-controlled Phase 2 gateway; direct provider bypass remains disabled. Unsupported questions abstain. Source endpoints resolve within the selected analysis; recommendations remain distinct.

Future free-form generative prose needs claim/source validation and an evaluation corpus. Never append unrestricted model text to factual output.

12. API and worker

All routes use `/api/v1`. Groups: projects; applications; artifacts; analyses/status/summary; class/package/domain graphs; domains/details; issues; metrics; cycles; modernization/sequence; external dependencies; assistant/query; evidence; JSON export; demo; manuals. `/docs` and `/openapi.json` describe contracts and query bounds.

Important errors: 404 missing entity, 409 unfinished results or missing artifact, 422 invalid input, 429 full queue, 503 unavailable scanner.

ThreadPoolExecutor(max_workers=1) executes jobs. A process-local admission lock permits four queued/running jobs. Stage updates are committed durably. The complete snapshot and COMPLETED status commit atomically. Startup requeues interrupted queued/running jobs when the original artifact remains available; missing inputs fail with recovery instructions. Failed scans can be retried as a new analysis. Run one API/worker process only.

Before multiple API workers/replicas, implement durable job leases, heartbeats, cancellation, idempotent retries and tenant capacity. The current lock is not distributed. Use analysis_id, stage duration, failure stage and counts for observability.

13. Testing

```
.\.runtime\python\tools\python.exe -m pytest backend/tests -q
$env:ARCHINTEL_RUN_SCANNER_TESTS='1'
.\.runtime\python\tools\python.exe -m pytest backend/tests/test_scanner_integration.py -q -p no:cacheprovider --basetemp=.data/fresh-test-directory
```

```
cd frontend
npm test
npm run build
npm exec playwright test
```

Use a fresh owned --basetemp directory when switching Windows execution accounts. Pytest may clean that directory; never use a general storage folder. The browser configuration uses installed Chrome; adjust executablePath for another machine.

Thirty initial backend tests cover algorithms, isolation, arithmetic, domain/scoring logic, assistant grounding, migration, upload security, failures and cleanup. Three opt-in tests exercise actual JAR/WAR/Boot scans with dead outbound HTTP proxy settings. Four frontend tests cover workspace/upload/assistant behavior. Two live Chrome tests exercise the investor journey and a fresh WAR upload. Consult the verification report for final results; test code alone is not proof of success.

14. Improvement priorities

1. Authentication, authorization, true tenant isolation, audit trails, encryption and backups.
2. Dedicated OS/container-sandboxed scanner workers with CPU/memory/disk limits and enforced egress controls. Avoiding application execution is not equivalent to OS sandboxing.
3. Durable queue, cancellation, retries, restart-safe leases and idempotency.
4. Abandoned-upload/crash cleanup and configurable enterprise retention.
5. Indexed normalized results, pagination for all large collections, and measured capacity limits.
6. A pinned real-world corpus: compatible PetClinic, Struts, Broadleaf, Java versions and packaging variants. They have not been validated merely because the synthetic corpus passes.
7. Better classpath resolution, resource/configuration analysis, framework wiring and generated-code handling.
8. Manual/hybrid domains with provenance and reproducible re-scoring.
9. Calibrated readiness, incoming pressure/boundary penalties and architecture-style-specific rules.
10. Evaluated generative providers and stronger user-facing source navigation.
11. Security-patched runtimes and recurring dependency scanning before external distribution.

Keep every new feature at least as evidence-grounded as the current prototype. Configured safety bounds do not certify 20,000-class performance, and local demo success does not establish production readiness.

Dependability checkpoint: restart and browser boundary

The worker atomically claims a `QUEUED` analysis with a conditional SQL update. Duplicate submissions do not execute or clean up another attempt. At startup it requeues persisted `QUEUED`/`RUNNING` analyses whose artifacts remain available; missing inputs produce `RECOVERY_FAILED`. Each attempt uses a new UUID-suffixed directory below `.data/jobs`, preventing reuse of a partially written Neo4j store. SHA-256 is checked before scanning. Failed inputs remain available for a new analysis; successful inputs follow `RETAIN_ARTIFACTS`.

Run exactly one API/worker process. This is not a distributed lease implementation: a second instance must not invoke recovery while another instance is scanning. Crash-leftover job directories and old failed artifacts still need managed retention; do not delete files used by a running scanner. Cancellation, backoff policies, process fencing, and durable multi-worker scheduling remain planned.

`app/security.py` rejects untrusted browser origins on API requests, including cross-site requests without Origin when browser fetch metadata identifies them. TrustedHostMiddleware validates Host before origin comparison. Normal CLI/OTel requests without browser headers remain supported, and OTLP retains its existing bearer-token authentication. This is browser-origin protection, not user authentication. Keep the service on localhost until account authorization and workspace isolation are implemented. Configure `ALLOWED_HOSTS` and `CORS_ORIGINS` explicitly for supported deployment changes; do not use wildcard hosts.

Tests in `backend/tests/test_dependability.py` cover persisted recovery, missing inputs, failed-scan retry, checksum rejection, duplicate claims, and origin/host rejection. The remaining product roadmap is in `product-completion-plan.md`.